

نظام إدارة خدام الكنيسة (CSMS)

الخطة الفنية الموحدة: النتائج + الحضور

الإصدار: 1.0

التاريخ: 27 ديسمبر 2025

تم الإعداد لـ: قيادة الكنيسة + فريق التطوير

الجدول الزمني: 20-26 أسبوعاً (5-6 أشهر) | غير مستعجل، التركيز على الجودة

الحالة: جاهز للتنفيذ

الملخص التنفيذي

تحويل نظامين منفصلين ومفككين إلى تطبيق محمول واحد موحد، يعمل أولاً بدون اتصال بالإنترنت (Offline-First)، وآمن، مع واجهة خلفية (Backend) قابلة للتوسع. تعالج هذه الخطة الشاملة الثغرات الأمنية الحرجة، وتحديات توزيع التطبيق، ومشاكل موثوقية النظام، مع وضع أساس للنمو المستدام.

- **الوضع الحالي:** مشروعان منفصلان، ثغرات أمنية كبيرة، كابوس في التوزيع اليدوي للتطبيق.
- **الوضع المستهدف:** تطبيق محمول واحد موحد، يعمل بدون اتصال، آمن، مع واجهة خلفية قابلة للتوسع باستخدام Firebase.
- **الاستثمار المطلوب:** خبرة تقنية + المستوى المجاني من Firebase (من 0 إلى 300 دولار/شهرياً عند التوسع الكبير).
- **الجدول الزمني المتوقع:** 20-26 أسبوعاً (5-6 أشهر)، تطوير غير مستعجل يركز على الجودة.

الجزء 1: تحليل المشاكل الحرجة

1.1 الثغرات الأمنية

المشكلة 1: نتائج الطلاب مكشوفة (🔴 حرجة)

مشاكل التنفيذ الحالي:

- لا يلزم تسجيل الدخول—يقوم المستخدمون بإدخال رقم الهوية فقط.
- يمكن لأي شخص تخمين هوية زميله والاطلاع على درجاته.
- لا يوجد سجل تدقيق (Audit Trail)—من المستحيل تحديد من وصل إلى أي بيانات.
- غياب كامل لضوابط الوصول.

التأثير الواقعي:

- انتهاك خصوصية الطلاب.
- لا يمكن إثبات نزاهة البيانات.
- يخالف متطلبات حوكمة بيانات الكنيسة.
- مسؤولية قانونية وتنظيمية محتملة.

الحل المقترح:

المصادقة باستخدام اسم المستخدم وكلمة المرور مع جلسات تعتمد على JWT، والتحكم في الوصول القائم على الأدوار (Role-based access control)، وتسجيل شامل لجميع عمليات الوصول للبيانات.

المشكلة 2: نقاط فشل فردية في الحضور (● عالية)

مشاكل التنفيذ الحالي:

- مسؤول واحد فقط (Admin) يمكنه تسجيل الحضور.
- فشل النظام في حال غياب المسؤول الأساسي.
- لا توجد آلية موزعة للتحكم في الوصول.
- عنق زجاجة تشغيلي يؤثر على تنفيذ الخدمة.

التأثير الواقعي:

- خطر تعطل الخدمة.
- اعتماد غير عادل على فرد واحد.
- عدم القدرة على التوسع لخدام إضافيين.
- شلل النظام أثناء غياب الموظفين.

الحل المقترح:

تحكم في الوصول قائم على الأدوار لتمكين عدة خدام من تسجيل الحضور، وهندسة نظام موزعة، وإدارة صلاحيات قابلة للتوسع.

المشكلة 3: عملية التوزيع اليدوية (● عالية)

مشاكل التنفيذ الحالي:

- يتم توزيع ملف Android APK يدوياً لكل جهاز.
- كل تحديث للتطبيق يتطلب إعادة توزيع يدوية.
- تشتت الإصدارات (Fragmentation) عبر النسخ المثبتة.
- فشل التحديثات بصمت دون أي رؤية.

التأثير الواقعي:

- الميزات غير متاحة في الإصدارات القديمة.
- التحديثات الأمنية لا تصل إلى جميع الأجهزة.
- استثمار كبير للوقت في التوزيع اليدوي.
- حالة تطبيق غير منضبطة عبر النشر.

الحل المقترح:

التوزيع التلقائي عبر متجر Google Play مع Firebase Remote Config لطرح الميزات تدريجياً والتصحيح الأمني التلقائي.

المشكلة 4: فشل العمل بدون اتصال (● عالية)

مشاكل التنفيذ الحالي:

- يتطلب تطبيق الهاتف اتصالاً مستقراً بالشبكة.
- البنية التحتية لوأي فاي الكنيسة غير مستقرة.
- فقدان بيانات الحضور عند انقطاع الاتصال.
- لا توجد آلية للتخزين المؤقت أو المزامنة بدون اتصال.

التأثير الواقعي:

- فقدان البيانات خلال ساعات الخدمة.
- إحباط المستخدمين وانخفاض التبني.
- نظام غير موثوق به أثناء خدمات الكنيسة الحرجة.
- سجلات حضور غير مكتملة.

الحل المقترح:

بنية تعتمد على العمل بدون اتصال أولاً (Offline-first) مع تخزين محلي باستخدام Hive، ومزامنة تلقائية عند استعادة الاتصال، وآليات لحل التعارضات.

1.2 أوجه القصور المعماري

المشكلة المعمارية	الوضع الحالي	التأثير	الحل
لا يوجد نموذج بيانات موحد	قاعدتا بيانات منفصلتان	كابوس المزامنة اليدوية	Firestore كمصدر وحيد للحقيقة
لا إدارة للجلسات	بحث عن الهوية (ID) بدون حالة	ثغرة أمنية	JWT + مصادقة Firebase
لا وصول قائم على الأدوار	المسؤول مقابل المستخدم غير واضح	زحف الصلاحيات	قواعد أمان Firestore + مصفوفة الأدوار
لا استراتيجية مزامنة	لا يوجد دعم للعمل دون اتصال	فقدان البيانات	Hive (محلي) + Firestore (سحابي)
لا معالجة للأخطاء	التطبيق يتعطل بصمت	فشل نظام غير معروف	معالجة شاملة للأخطاء + تقارير
لا تحليلات	لا يمكن تتبع أنماط الاستخدام	قرارات نشر عمياء	تحليلات Firebase مدمجة

الجزء 2: بنية النظام المقترحة

2.1 تصميم النظام عالي المستوى

تدمج البنية الموحدة المقترحة جميع الوظائف في نظام واحد متماسك مع فصل واضح للاهتمامات عبر طبقات الواجهة الأمامية، والخلفية، والتكامل.

طبقة الواجهة الأمامية (تطبيق الهاتف المحمول - Flutter):

- وحدة النتائج: عرض جميع الفترات الدراسية (Terms) مع السياق التاريخي.
- وحدة الحضور: التسجيل اليدوي، مسح الباركود، جاهزية للتعرف على الوجه (Face ID).
- لوحة تحكم المسؤول: تقارير شاملة وإدارة المستخدمين.
- بنية Offline-First: تخزين محلي يعتمد على Hive مع مزامنة تلقائية.

طبقة الواجهة الخلفية (Firestore + خدمات سحابية):

- المصادقة: Firebase Auth مع تحكم في الوصول قائم على الأدوار.
- قاعدة البيانات: Firestore مع قواعد أمان دقيقة.

- **منطق العمل:** Cloud Functions للعمليات المعقدة.
- **التخزين:** Firestore و Cloud Storage للمستندات والصور.
- **تحديثات في الوقت الفعلي:** مستمعو Firestore لمزامنة واجهة المستخدم تلقائياً.

التكاملات الخارجية:

- **Google Play Store:** توزيع التطبيق تلقائياً.
- **Google Play App Signing:** بنية تحتية للأمان.
- **Firebase Crashlytics:** مراقبة الإنتاج وتتبع الأخطاء.

2.2 وحدات النظام الأساسية

الوحدة 1: المصادقة وإدارة المستخدمين

- **المسؤولية:** مصادقة آمنة، تحكم في الوصول قائم على الأدوار، وإدارة الجلسات.
- **المكونات الرئيسية:** مصادقة Firebase، رموز JWT مخصصة، ملفات تعريف الخدام، مصفوفة الأدوار.
- **تنفيذ الأمان:** تشفير كلمات المرور (bcrypt)، انتهاء صلاحية JWT كل 24 ساعة، تشفير البيانات الحساسة.

الوحدة 2: نظام إدارة النتائج

- **المشكلة الحالية:** رؤية لفترة واحدة فقط، لا سياق تاريخي، لا ضوابط وصول.
- **التصميم المقترح:** تاريخ متعدد الفترات، جدول زمني مرئي، تحكم في الوصول لكل خادم.

الوحدة 3: نظام إدارة الحضور

- **المشكلة الحالية:** عنق زجاجة لمسؤول واحد، توزيع يدوي صعب.
- **التصميم المقترح:** تشغيل Offline-first، قدرة لعدة خدام، تعقيد تدريجي للميزات.
- **مراحل التنفيذ:**

- المرحلة 1 (v1.0): الاختيار اليدوي + مسح الباركود .
- المرحلة 2 (v1.5): التعرف على الوجه اختياري.
- المرحلة 3 (مستقبلية): نهج هجين.

الوحدة 4: لوحة تحكم المسؤول (Admin Dashboard)

- **المسؤوليات الأساسية:** إنشاء وإدارة حسابات المستخدمين، رفع ونشر النتائج، تقارير الحضور، مراقبة حالة المزامنة.
- **الميزات الرئيسية:** استيراد CSV للنتائج، واجهة إدخال يدوي بديلة، تصدير PDF/CSV، مراقبة صحة النظام.

الجزء 3: مبررات اختيار التقنيات

المكون	التقنية	السبب (Rationale)	البديل
الواجهة الأمامية	Flutter	قاعدة كود واحدة (iOS + Android)	React Native
تخزين الهاتف	Hive	Offline-first، لا SQL، سريع	SQLite, Realm
قاعدة البيانات	Firestore	الوقت الفعلي، العمل دون اتصال، قواعد أمان	Firebase RT DB
المصادقة	Firebase Auth	عديمة الحالة (Stateless)، آمنة	JWT مخصص
المزامنة	Firestore	تلقائية، خالية من التعارضات	REST polling
التوزيع	Google Play	تحديثات تلقائية، أمان	APK يدوي
مسح الباركود	flutter_barcode_scanner	بسيط، موثوق	google_mlkit
المراقبة	Firebase Crashlytics	تتبع تلقائي	Sentry

الجزء 4: مبررات البنية الموحدة للنظام

لماذا الدمج في تطبيق واحد؟

1. تجربة المستخدم: النتائج + الحضور في تطبيق واحد (لا فقدان للسياق).
2. المزامنة دون اتصال: محرك مزامنة واحد، مصدر واحد للحقيقة.
3. تحديثات التطبيق: ملف APK واحد للصيانة والتوزيع.
4. اتساق البيانات: قاعدة بيانات واحدة، مخطط موحد.
5. نموذج الصلاحيات: وصول دقيق لكل ميزة.

توصية استراتيجية: تطبيق واحد موحد لأن الخدام هم نفس المستخدمين الذين يحتاجون إلى كلتا الوظيفتين.

الجزء 5: خارطة طريق التنفيذ (20-26 أسبوعاً)

المرحلة 1: الأساس والأمان (الأسابيع 1-8)

- الهدف الاستراتيجي: إنشاء مصادقة آمنة، قدرة Offline-first، وبنية تحتية لتحويل البيانات.
- التسليمات: تسجيل دخول آمن، تخزين محلي مع مزامنة، تحويل البيانات القديمة، تغطية اختبار شاملة.

المرحلة 2: الميزات الأساسية (الأسابيع 9-18)

- 2A: وحدة النتائج (الأسابيع 9-12): عرض سجل النتائج الكامل، وظائف دون اتصال 100%، مزامنة تلقائية.
- 2B: وحدة الحضور - المرحلة 1 (الأسابيع 13-18): تسجيل حضور متعدد الخدام، مسح الباركود، حل التعارضات، تقارير الحضور.

المرحلة 3: ميزات المسؤول والنشر للإنتاج (الأسابيع 19-22)

- الهدف الاستراتيجي: لوحة تحكم المسؤول، نشر النتائج، إطلاق Google Play.
- التسليمات: رفع النتائج، إدارة المستخدمين، التطبيق على المتجر، التقارير.

المرحلة 4: الميزات المتقدمة والتحسين (الأسابيع 23-26+)

- الهدف: تحسين الأداء، ميزات متقدمة (Face ID)، تدقيق أمني.
- التسليمات: تطبيق جاهز للإنتاج، إشعارات، تحليلات.

الجزء 6: التأثير الواقعي وحل المشكلات

- ✓ مشكلة 1: انتهاك خصوصية الطلاب - تم الحل بمصادقة آمنة.
- ✓ مشكلة 2: الاعتماد على مسؤول واحد - تم الحل بتوزيع الصلاحيات.
- ✓ مشكلة 3: موثوقية العمل دون اتصال - تم الحل بـ Hive و Firestore.
- ✓ مشكلة 4: تعقيد التوزيع - تم الحل عبر Google Play.
- ✓ مشكلة 5: نزاهة البيانات - تم الحل بسجلات التدقيق.
- ✓ مشكلة 6: قيود البيانات التاريخية - تم الحل بالسجل التاريخي الكامل.

الجزء 7: أفضل ممارسات التنفيذ

- اختبار Offline-First: اختبر دائماً وظائف عدم الاتصال أولاً.
- Firebase Emulator: استخدم المحاكى للاختبار.
- الطرح التدريجي: النشر للمسار التجريبي قبل الإنتاج.

- قائمة التحقق الأمنية: قواعد أمان، تدوير مفاتيح، سجلات تدقيق.

الجزء 8: تقييم المخاطر واستراتيجيات التخفيف

عامل الخطر	الأثر المحتمل	استراتيجية التخفيف
فقدان بيانات Firestore	فشل كامل للنظام	نسخ احتياطي تلقائي يومي
انتهاء صلاحية الرمز (Offline)	منع دخول المستخدم	فترة سماح 24 ساعة
تعارض المزامنة	تلف البيانات	حل يدوي + "آخر كتابة تفوز"
رفض Google Play	تأخير الإطلاق	تقديم مبكر

الجزء 9: أسئلة توضيحية حرجة

قبل بدء التطوير، يجب معالجة هذه الأسئلة مع قيادة الكنيسة:

1. جدوى الجدول الزمني: هل 5-6 أشهر واقعية؟
2. أولوية **Face ID**: هل نؤجلها للإصدار v1.5؟
3. دعم اللغة: عربية فقط أم واجهة مزدوجة؟
4. صلاحيات التقارير: من يحق له الوصول؟
5. تنسيقات التصدير: CSV أم PDF؟
6. الاحتفاظ بالبيانات: كم المدة؟
7. تكاليف السحابة: هل هي مقبولة؟

الخلاصة

تعالج هذه البنية التقنية الموحدة بشكل شامل كل مشكلة تم تحديدها في العالم الحقيقي:

- ☒ الأمان: مصادقة قوية.

- **Offline-First** ✓ عمل كامل دون اتصال.
- **المزامنة** ✓ متعددة الأجهزة.
- **التوزيع** ✓ تلقائي عبر المتجر.
- **المساءلة** ✓ سجلات تدقيق شاملة.

توصية استراتيجية: ابدأ المرحلة 1 هذا الأسبوع.

الخطوات التالية:

- مراجعة الخطة مع فريق التطوير وقيادة الكنيسة.
- إعداد مشروع Firebase.
- تهيئة بيئة تطوير Flutter.

إعداد: كيرلس + التحليل الفني (معاكم)